Level 2
Hackathon – Fundamentals of Web Development

Student Details

Student Name	S. Vithya Sri
Register Number	34625U50033
Department	BSC (AI)
ID	5609A4EA396428EB25E4D994E2717C7D
College	M.M.E.S Women's Art and Science College
Date of Submission	28/10/2026

Git-Hub Link:
https://github.com/vithyasri/Student-Attendance-Tracker

1. Problem Statement
Many colleges still record student attendance in paper registers or scattered spreadsheets. Manual entry is slow, mistakes and duplicate entries are common, attendance percentages must be calculated by hand, and students who fall below the required percentage are noticed too late.

A simple, responsive web application is therefore needed that lets staff maintain student records, mark daily attendance, calculate attendance percentages automatically and instantly identify students with low attendance.

2. Objectives
- Add, edit and delete student details (name, student ID, department).
- Mark attendance by date as Present, Absent, Late or Leave, without duplicate entries for the same student and date.
- Calculate attendance percentage dynamically for every student and for the whole class.
- Search and filter students by name, ID, department or attendance status.
- Show student-wise attendance history and highlight students below a configurable threshold.
- Store all data in the browser using Local Storage so it is available after a page refresh.

Expected outcome: a fast, easy-to-use tracker that reduces manual work, avoids errors and helps staff take early action for students with poor attendance.

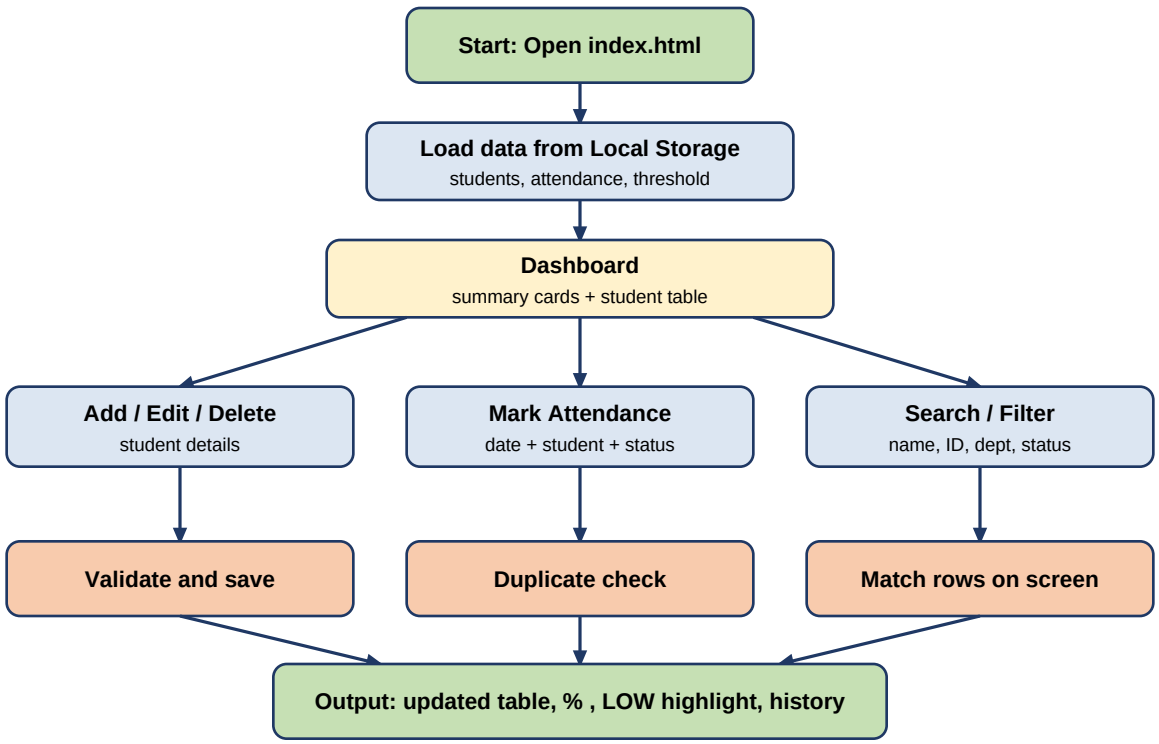
3. Architecture
The application is a front-end only, single-page web app. **index.html** defines the layout, **style.css** handles the design and **script.js** contains all logic. There is no server; data is saved in the browser's Local Storage.

Technologies used

- **HTML5** – structure of the page: summary cards, forms, filters, table and history area.
- **CSS3** – styling, Grid/Flexbox layout, colour-coded status badges and responsive design (media queries).
- **JavaScript (ES6)** – DOM handling, events, attendance calculation, search/filter and validation.
- **Local Storage** – permanent storage of students, attendance records and threshold value.

User navigation flow: The user opens the page and sees the attendance summary and student list. From the left panel the user adds/edits students and marks attendance; from the right panel the user searches, filters, opens a student's history or deletes a student. Every action saves to Local Storage and refreshes the screen immediately.

Application flow diagram



4. Practical Implementation

The project was developed using Visual Studio Code and tested in Google Chrome. First the page layout was written in HTML, then styled with CSS, and finally JavaScript was added for the logic. The student list and attendance records are kept as JavaScript arrays of objects, saved as JSON in Local Storage after every change, and the screen is re-rendered from this data.

4.1 Features Implemented

- **Student management** – add, edit and delete students; duplicate student IDs are rejected.
- **Mark attendance by date** – choose date, student and status (Present, Absent, Late, Leave).

- **Duplicate prevention** – a second entry for the same student and date is blocked with a warning.
- **Dynamic percentage** – (Present + Late) ÷ total marked days × 100, updated instantly.
- **Search and filter** – by name, student ID, department, attendance status or low attendance.
- **Attendance history** – date-wise record of each student with status badges.
- **Low attendance highlight** – rows below the configurable threshold turn red with a LOW tag.
- **Attendance summary** – total students, present, absent and overall attendance percentage.
- **Local Storage** – data stays saved after refreshing or closing the browser.
- **Responsive design** – layout adapts to laptop, tablet and mobile screens.

4.2 Source Code

HTML

HTML gives the application its structure: header with threshold input, four summary cards, the Add Student and Mark Attendance forms, the search/filter bar, the student table and the history area. Elements have IDs so JavaScript can access them.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Attendance Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="topbar">
    <h1>Student Attendance Tracker</h1>
    <label class="threshold">Low-attendance threshold (%)
      <input type="number" id="threshold" min="1" max="100" value="75">
    </label>
  </header>

  <main>
    <!-- Attendance summary -->
    <section class="cards">
      <div class="card"><span>Total Students</span><b id="sumTotal">0</b></div>
      <div class="card green"><span>Present</span><b id="sumPresent">0</b></div>
      <div class="card red"><span>Absent</span><b id="sumAbsent">0</b></div>
      <div class="card blue"><span>Overall Attendance</span><b id="sumPercent">0%</b></div>
    </section>

    <div class="layout">
      <aside>
        <!-- Add / edit student -->
        <section class="panel">
          <h2 id="formTitle">Add Student</h2>
          <input id="stuName" placeholder="Student name">
          <input id="stuId" placeholder="Student ID (e.g. S101)">
          <input id="stuDept" placeholder="Department">
          <div class="row">
            <button id="saveBtn" class="btn primary">Add Student</button>
            <button id="cancelBtn" class="btn hidden">Cancel</button>
          </div>
        </section>

        <!-- Mark attendance -->
        <section class="panel">
          <h2>Mark Attendance</h2>
          <input type="date" id="attDate">
          <select id="attStudent"></select>
          <select id="attStatus">
            <option>Present</option>
            <option>Absent</option>
            <option>Late</option>
            <option>Leave</option>
          </select>
```

```
          <button id="markBtn" class="btn primary">Mark Attendance</button>
          <p id="msg" class="msg"></p>
        </section>
      </aside>

      <section class="panel wide">
        <!-- Search and filters -->
        <h2>Student Records</h2>
        <div class="filters">
          <input id="search" placeholder="Search by name or ID">
          <select id="deptFilter"><option value="">All Departments</option></select>
          <select id="statusFilter">
            <option value="">All Status (selected date)</option>
            <option>Present</option><option>Absent</option>
            <option>Late</option><option>Leave</option>
            <option value="Not Marked">Not Marked</option>
            <option value="Low">Low Attendance</option>
          </select>
        </div>
        <table>
          <thead>
            <tr><th>ID</th><th>Name</th><th>Department</th><th>Status</th><th>Attendance %</th><th>Actions</th></tr>
          </thead>
          <tbody id="tableBody"></tbody>
        </table>

        <!-- Student-wise history -->
        <div id="historyBox" class="history hidden">
          <h3 id="historyTitle"></h3>
          <div id="historyList"></div>
        </div>
      </section>
    </div>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS styles the application: gradient header, card design, a two-column Grid layout, coloured status badges, red highlighting for low-attendance rows, and a media query that stacks the layout on small screens.

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: "Segoe UI", Arial, sans-serif; }
body { background: #eef2f7; color: #1f2937; }
.topbar { background: linear-gradient(90deg, #1e3a8a, #2563eb); color: #fff; padding: 14px 28px;
  display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px; }
.topbar h1 { font-size: 22px; }
.threshold { font-size: 14px; display: flex; align-items: center; gap: 8px; }
.threshold input { width: 70px; padding: 5px; border-radius: 6px; border: none; }
main { padding: 18px 28px; }

/* Summary cards */
.cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; margin-bottom: 16px; }
.card { background: #fff; border-radius: 10px; padding: 14px 18px; border-left: 5px solid #6b7280;
  box-shadow: 0 1px 4px rgba(0,0,0,.1); }
.card span { font-size: 13px; color: #6b7280; display: block; }
.card b { font-size: 26px; }
.card.green { border-color: #16a34a; } .card.red { border-color: #dc2626; } .card.blue { border-color: #2563eb; }

/* Layout */
.layout { display: grid; grid-template-columns: 320px 1fr; gap: 16px; align-items: start; }
.panel { background: #fff; border-radius: 10px; padding: 16px; margin-bottom: 16px; box-shadow: 0 1px 4px
    rgba(0,0,0,.1); }
.panel h2 { font-size: 17px; margin-bottom: 12px; color: #1e3a8a; }
.panel input, .panel select { width: 100%; padding: 9px; margin-bottom: 10px; border: 1px solid #cbd5e1;
  border-radius: 6px; font-size: 14px; }
.row { display: flex; gap: 8px; }

/* Buttons */
.btn { padding: 9px 14px; border: none; border-radius: 6px; cursor: pointer; font-size: 14px; background: #e5e7eb; }
.btn.primary { background: #2563eb; color: #fff; width: 100%; }
.btn.small { padding: 4px 9px; font-size: 12px; margin-right: 3px; }
.btn.edit { background: #fef3c7; } .btn.hist { background: #dbeafe; } .btn.del { background: #fee2e2; }
.hidden { display: none; }
```

```
/* Messages */
.msg { font-size: 13px; margin-top: 8px; min-height: 18px; font-weight: 600; }
.msg.ok { color: #15803d; } .msg.err { color: #b91c1c; }

/* Table */
.filters { display: flex; gap: 10px; margin-bottom: 10px; }
.filters input, .filters select { margin-bottom: 0 !important; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th { background: #1e3a8a; color: #fff; text-align: left; padding: 9px; }
td { padding: 8px 9px; border-bottom: 1px solid #e5e7eb; }
tr.low { background: #fee2e2; }
.badge { padding: 2px 9px; border-radius: 12px; font-size: 12px; font-weight: 600; background: #e5e7eb; }
.badge.Present { background: #dcfce7; color: #166534; } .badge.Absent { background: #fecaca; color: #991b1b; }
.badge.Late { background: #fef3c7; color: #92400e; } .badge.Leave { background: #dbeafe; color: #1e40af; }
.low-tag { color: #b91c1c; font-weight: 700; font-size: 12px; margin-left: 6px; }

/* History */
.history { margin-top: 16px; border-top: 2px dashed #cbd5e1; padding-top: 12px; }
.history h3 { margin-bottom: 8px; color: #1e3a8a; }
.chip { display: inline-block; margin: 3px; padding: 4px 10px; border-radius: 6px; background: #f1f5f9; font-size:
    13px; }

@media (max-width: 800px) {
  .layout { grid-template-columns: 1fr; }
  .cards { grid-template-columns: repeat(2, 1fr); }
  .filters { flex-direction: column; }
  table { font-size: 12px; }
}
```

JavaScript

JavaScript implements the logic: loading and saving data in Local Storage, add/edit/delete, attendance marking with duplicate check, percentage calculation, search and filters, history display and re-rendering of the summary and table.

```
// ---------- Data (Local Storage) ----------
let students  = JSON.parse(localStorage.getItem("sat_students")   || "[]");
let records   = JSON.parse(localStorage.getItem("sat_attendance") || "[]"); // {sid, date, status}
let threshold = Number(localStorage.getItem("sat_threshold")) || 75;
let editingId = null;

const $ = id => document.getElementById(id);
const today = new Date().toISOString().slice(0, 10);

function save() {
  localStorage.setItem("sat_students", JSON.stringify(students));
  localStorage.setItem("sat_attendance", JSON.stringify(records));
  localStorage.setItem("sat_threshold", threshold);
}
function showMsg(text, type) { $("msg").textContent = text; $("msg").className = "msg " + type; }
function esc(t) { const d = document.createElement("div"); d.textContent = t; return d.innerHTML; }

// ---------- Attendance percentage ----------
function percent(sid) {
  const list = records.filter(r => r.sid === sid);
  if (!list.length) return null;
  const attended = list.filter(r => r.status === "Present" || r.status === "Late").length;
  return Math.round((attended / list.length) * 100);
}
const statusOn = (sid, date) => (records.find(r => r.sid === sid && r.date === date) || {}).status || "Not Marked";
const isLow = sid => percent(sid) !== null && percent(sid) < threshold;

// ---------- Add / Edit / Delete students ----------
function saveStudent() {
  const name = $("stuName").value.trim(), id = $("stuId").value.trim(), dept = $("stuDept").value.trim();
  if (!name || !id || !dept) return alert("Please fill all student details.");
  if (students.some(s => s.id === id && s.id !== editingId)) return alert("Student ID already exists!");
  if (editingId) {
    const s = students.find(x => x.id === editingId);
    records.forEach(r => { if (r.sid === editingId) r.sid = id; });
    Object.assign(s, { name, id, dept });
  } else {
    students.push({ name, id, dept });
  }
  resetForm(); save(); render();
}
```

```
function editStudent(id) {
  const s = students.find(x => x.id === id);
  editingId = id;
  $("stuName").value = s.name; $("stuId").value = s.id; $("stuDept").value = s.dept;
  $("formTitle").textContent = "Edit Student";
  $("saveBtn").textContent = "Update Student";
  $("cancelBtn").classList.remove("hidden");
}
function resetForm() {
  editingId = null;
  ["stuName", "stuId", "stuDept"].forEach(i => $(i).value = "");
  $("formTitle").textContent = "Add Student";
  $("saveBtn").textContent = "Add Student";
  $("cancelBtn").classList.add("hidden");
}
function deleteStudent(id) {
  if (!confirm("Delete this student and all attendance records?")) return;
  students = students.filter(s => s.id !== id);
  records = records.filter(r => r.sid !== id);
  $("historyBox").classList.add("hidden");
  save(); render();
}

// ---------- Mark attendance (no duplicates) ----------
function markAttendance() {
  const sid = $("attStudent").value, date = $("attDate").value, status = $("attStatus").value;
  if (!sid || !date) return showMsg("Select a student and a date.", "err");
  if (records.some(r => r.sid === sid && r.date === date))
    return showMsg("Attendance already marked for this student on " + date + ".", "err");
  records.push({ sid, date, status });
  save(); render();
  showMsg("" + status + " marked successfully.", "ok");
}

// ---------- History ----------
function showHistory(sid) {
  const s = students.find(x => x.id === sid);
  const list = records.filter(r => r.sid === sid).sort((a, b) => b.date.localeCompare(a.date));
  $("historyTitle").textContent = "Attendance History – " + s.name + " (" + (percent(sid) ?? 0) + "%)";
  $("historyList").innerHTML = list.length
    ? list.map(r => `<span class="chip">${r.date} – <span class="badge
        ${r.status}">${r.status}</span></span>`).join("")
    : "No attendance records yet.";
  $("historyBox").classList.remove("hidden");
}

// ---------- Render UI ----------
function render() {
  const date = $("attDate").value || today;
  const q = $("search").value.toLowerCase(), dept = $("deptFilter").value, st = $("statusFilter").value;

  // summary
  const present = students.filter(s => ["Present", "Late"].includes(statusOn(s.id, date))).length;
  const absent  = students.filter(s => statusOn(s.id, date) === "Absent").length;
  const attended = records.filter(r => r.status === "Present" || r.status === "Late").length;
  $("sumTotal").textContent = students.length;
  $("sumPresent").textContent = present;
  $("sumAbsent").textContent = absent;
  $("sumPercent").textContent = (records.length ? Math.round(attended / records.length * 100) : 0) + "%";

  // dropdowns
  const depts = [...new Set(students.map(s => s.dept))];
  $("deptFilter").innerHTML = '<option value="">All Departments</option>' +
    depts.map(d => `<option ${d === dept ? "selected" : ""}>${esc(d)}</option>`).join("");
  const chosen = $("attStudent").value;
  $("attStudent").innerHTML = '<option value="">-- Select Student --</option>' +
    students.map(s => `<option value="${esc(s.id)}" ${s.id === chosen ? "selected" : ""}>${esc(s.id)} -
        ${esc(s.name)}</option>`).join("");

  // filtered table
  const rows = students.filter(s => {
    const status = statusOn(s.id, date);
    return (s.name.toLowerCase().includes(q) || s.id.toLowerCase().includes(q)) &&
      (!dept || s.dept === dept) &&
      (!st || (st === "Low" ? isLow(s.id) : status === st));
  });
  $("tableBody").innerHTML = rows.length ? rows.map(s => {
    const p = percent(s.id), status = statusOn(s.id, date);
    return `<tr class="${isLow(s.id) ? "low" : ""}">
      <td>${esc(s.id)}</td><td>${esc(s.name)}</td><td>${esc(s.dept)}</td>
```

```
      <td><span class="badge ${status}">${status}</span></td>
      <td>${p === null ? "–" : p + "%"}${isLow(s.id) ? '<span class="low-tag">LOW</span>' : ""}</td>
      <td>
        <button class="btn small edit" onclick="editStudent('${esc(s.id)}')">Edit</button>
        <button class="btn small hist" onclick="showHistory('${esc(s.id)}')">History</button>
        <button class="btn small del" onclick="deleteStudent('${esc(s.id)}')">Delete</button>
      </td></tr>`;
  }).join("") : '<tr><td colspan="6">No students found.</td></tr>';
}

// ---------- Events ----------
$("saveBtn").onclick = saveStudent;
$("cancelBtn").onclick = resetForm;
$("markBtn").onclick = markAttendance;
$("attDate").value = today;
$("attDate").onchange = render;
$("threshold").value = threshold;
$("threshold").oninput = e => { threshold = Number(e.target.value) || 75; save(); render(); };
["search", "deptFilter", "statusFilter"].forEach(i => $(i).oninput = render);
render();
```

4.3 Project Execution

How to run:

- Keep **index.html**, **style.css** and **script.js** in the same folder.
- Double-click **index.html** (or use Live Server in VS Code) to open it in a browser such as Chrome.
- No installation or internet connection is required.

Working process: The user first adds students using the Add Student form. Then a date, a student and a status are selected and *Mark Attendance* is clicked. JavaScript checks for a duplicate record, saves the entry to Local Storage, recalculates the percentage and refreshes the summary cards and table. The user can search or filter the list, open a student's history, edit or delete a record, and change the threshold to highlight students with low attendance.

5. Output Screenshots

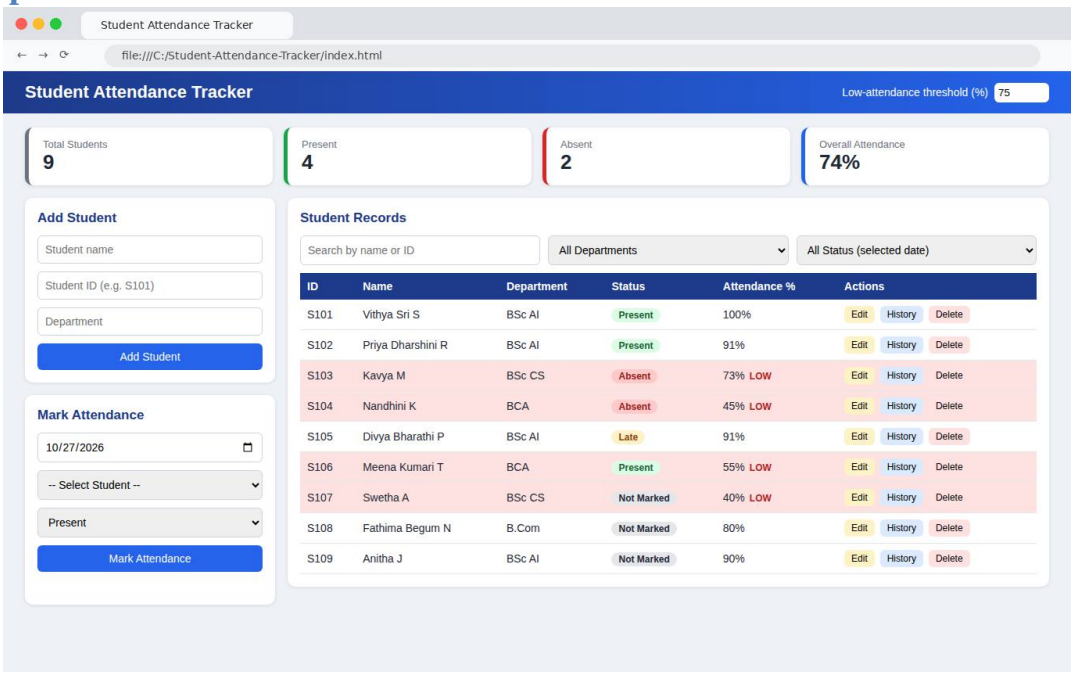


Figure 1: Home page with attendance summary, student records and low-attendance highlighting

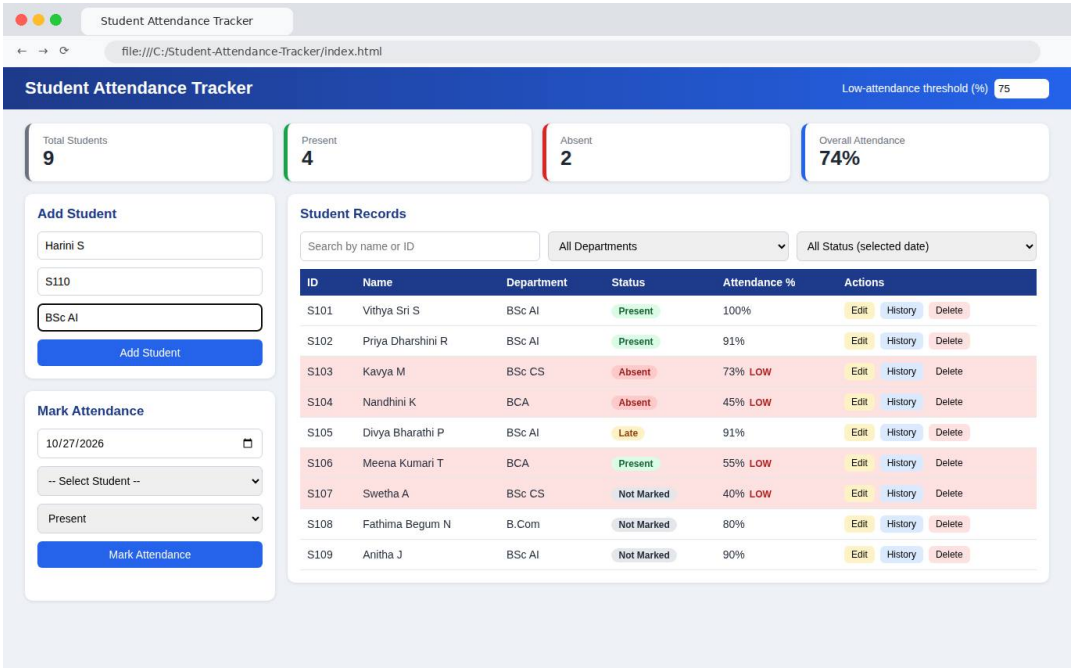


Figure 2: Add Student form filled with new student details

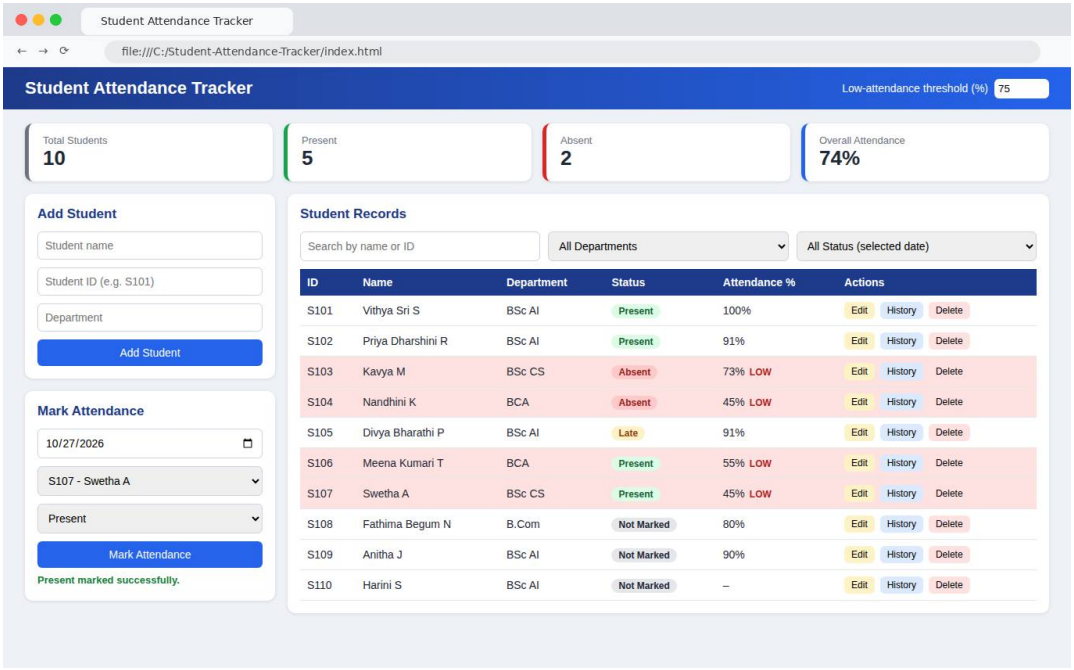


Figure 3: Attendance marked successfully; table and summary cards updated

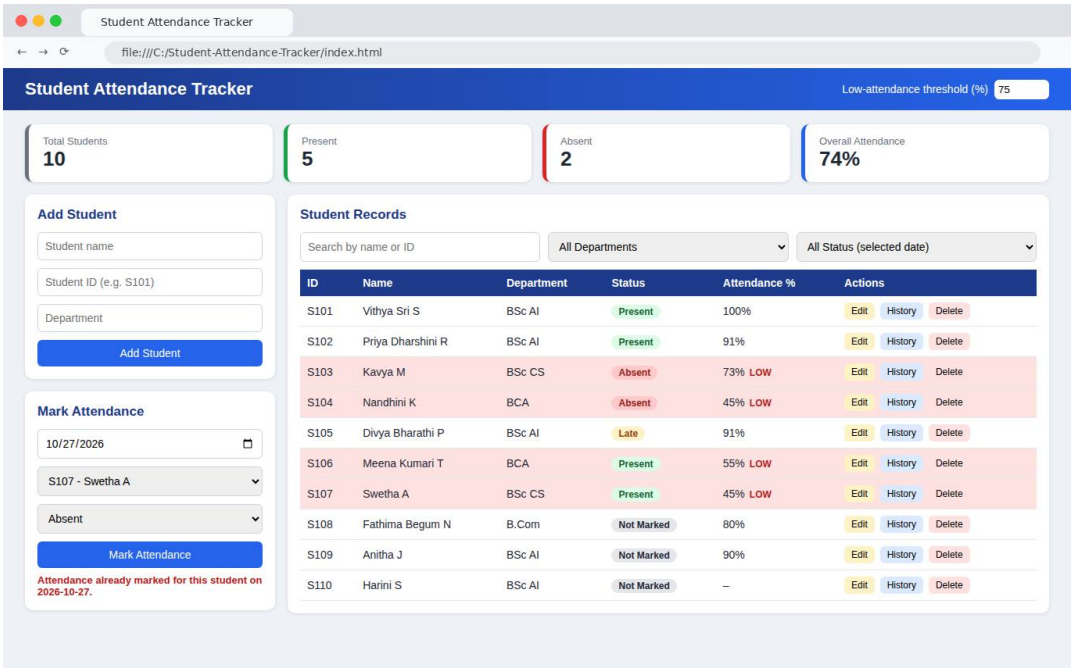


Figure 4: Duplicate attendance entry for the same student and date is blocked

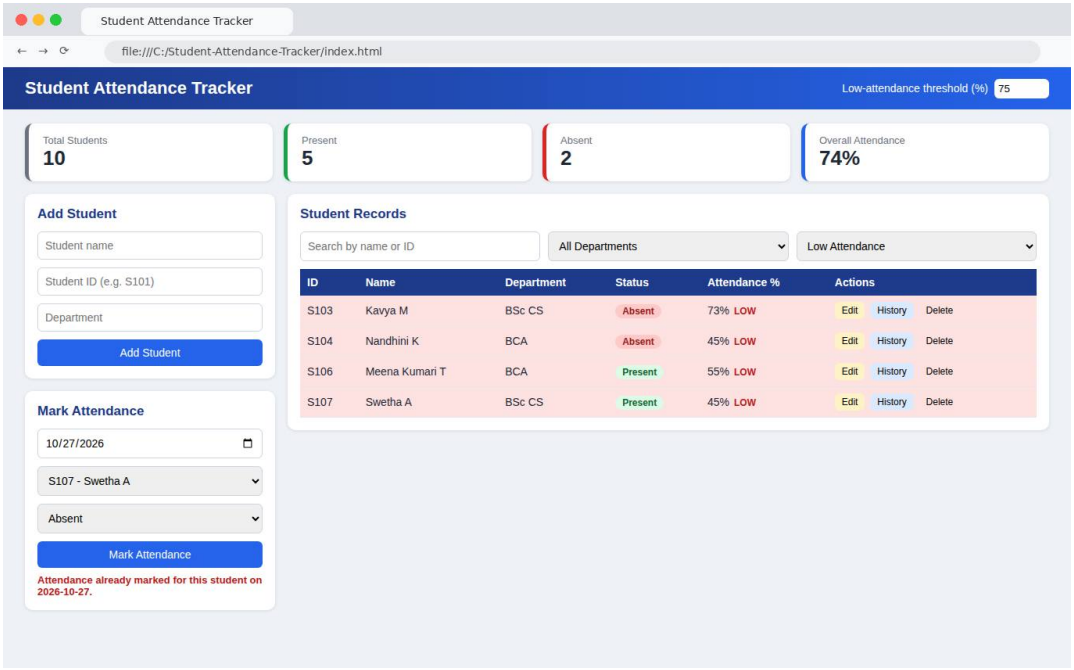


Figure 5: Low Attendance filter showing students below the 75% threshold

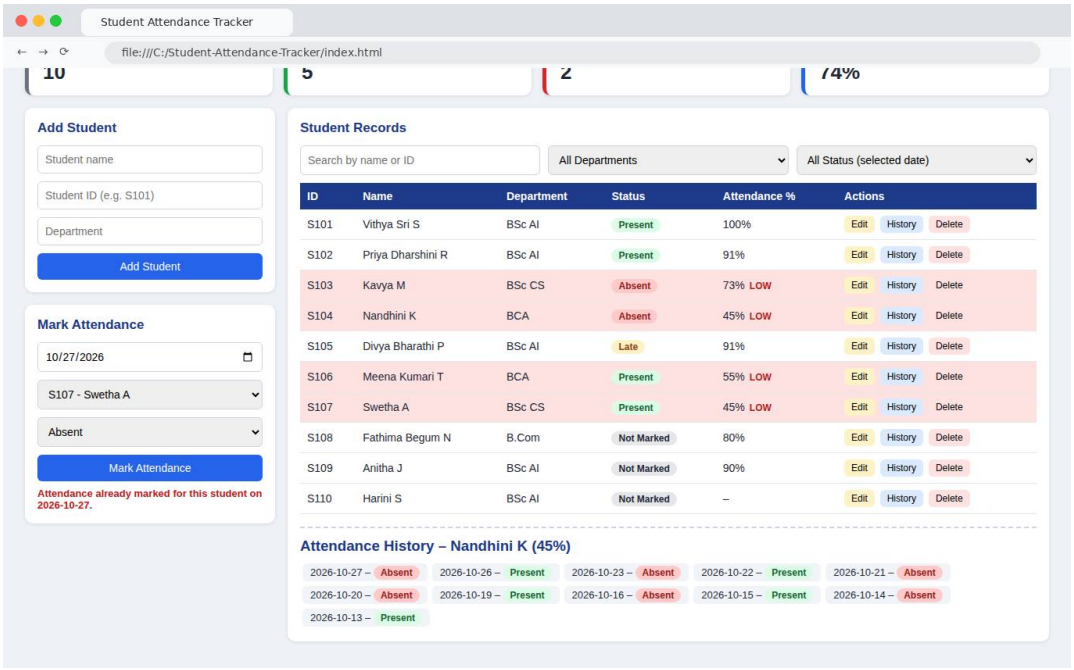


Figure 6: Student-wise attendance history with percentage

6. Technical Explanation

- **HTML5** – semantic structure; form controls (text, date, select) collect user input.

- **CSS3** – Grid and Flexbox for layout, class-based styling for status badges, media queries for responsiveness.
- **DOM manipulation** – getElementById, innerHTML and textContent update the page without reloading.
- **Event handling** – onclick, oninput and onchange trigger add, mark, search and filter actions.
- **Arrays and objects** – students are stored as {name, id, dept}; attendance as {sid, date, status}. filter(), find(), some() and map() are used for searching, duplicate checking and rendering.
- **Local Storage with JSON** – JSON.stringify() saves data and JSON.parse() restores it on page load.
- **Percentage logic** – percentage = (Present + Late) ÷ total records × 100; a student is flagged LOW when it is below the threshold.
- **Validation** – empty fields, duplicate student IDs and duplicate attendance entries are rejected.

7. Conclusion

The Student Attendance Tracker was successfully developed using HTML, CSS and JavaScript. It allows students to be added, edited and deleted, attendance to be marked without duplicates, percentages to be calculated automatically, and low-attendance students to be identified quickly. Through this project I learned DOM manipulation, event handling, array methods, Local Storage, responsive design and input validation. The application works fully in the browser and meets all the expected features.

8. Future Enhancements

- Login system for teachers and administrators, with secure password handling.
- Backend and database (Node.js/MySQL or Firebase) for multi-device, multi-user access.
- Export of attendance reports to PDF or Excel and charts for monthly analysis.
- Automatic email or SMS alerts to students and parents for low attendance.
- Class-wise and subject-wise attendance, QR code or biometric attendance marking.
- Dark mode and improved mobile and accessibility support.